

08-二分基础

一、本阶段目标

二分是 CSP 中极其重要的算法思想。

这一章的目标是：

- 理解二分本质
- 掌握普通二分查找
- 掌握边界二分
- 掌握 `lower_bound` / `upper_bound`
- 掌握答案二分
- 理解 `check` 函数
- 理解“单调性”
- 熟悉常见二分模型

二分最核心的一句话：

1 每次排除一半答案。

二、为什么二分快？

普通查找：

1 一个一个试

复杂度：

1 $O(n)$

二分：

1 每次砍掉一半范围

复杂度：

```
1  O(log n)
```

例子

例如：

```
1  1~1000 中找一个数
```

普通查找：

```
1  最坏可能找 1000 次
```

二分：

```
1  10 次左右就够
```

因为：

```
1  2^10≈1024
```

三、二分的核心前提

1. 有序

普通二分查找必须：

```
1  数组有序。
```

例如：

```
1  sort(a, a+n);
```

2. 单调性

答案二分必须：

- 1 满足单调性。

也就是：

- 1 答案越大越容易成立

或者：

- 1 答案越小越容易成立

四、普通二分查找

1. 题目模型

例如：

- 1 在有序数组中找 x 是否存在。

2. 思路

每次：

- 1 看中间值 mid

然后：

- 1 目标在左边？
- 2 目标在右边？
- 3 已经找到？

3. 基础模板

```
1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4
```

```

5  int main()
6  {
7      int n,x;
8      cin>>n>>x;
9
10     int a[100];
11
12     for(int i=0;i<n;i++)
13     {
14         cin>>a[i];
15     }
16
17     sort(a,a+n);
18
19     int l=0,r=n-1;
20
21     while(l<=r)
22     {
23         int mid=(l+r)/2;
24
25         if(a[mid]==x)
26         {
27             cout<<"找到";
28             return 0;
29         }
30         else if(a[mid]<x)
31         {
32             l=mid+1;
33         }
34         else
35         {
36             r=mid-1;
37         }
38     }
39
40     cout<<"不存在";
41
42     return 0;
43 }

```

五、为什么这样移动？

1. mid 太小

如果：

```
1  a[mid]<x
```

说明：

```
1  mid 和左边都不可能是答案。
```

所以：

```
1  l=mid+1;
```

2. mid 太大

如果：

```
1  a[mid]>x
```

说明：

```
1  mid 和右边都不可能是答案。
```

所以：

```
1  r=mid-1;
```

六、边界二分

普通二分：

```
1  找一个值。
```

边界二分：

```
1  找边界位置。
```

例如：

- 1 第一个 $\geq x$ 的位置
 - 2 第一个 $> x$ 的位置
 - 3 最后一个 $\leq x$ 的位置
-

七、lower_bound 和 upper_bound

1. lower_bound

```
1 lower_bound(a.begin(), a.end(), x)
```

含义：

- 1 第一个 $\geq x$ 的位置
-

2. upper_bound

```
1 upper_bound(a.begin(), a.end(), x)
```

含义：

- 1 第一个 $> x$ 的位置
-

3. 一句话记忆

- 1 lower: 从 x 开始找
 - 2 upper: 从 x 上面开始找
-

4. 示例

```
1 vector<int> a={1,2,2,2,5};  
2  
3 int l=lower_bound(a.begin(),a.end(),2)-a.begin();  
4 int r=upper_bound(a.begin(),a.end(),2)-a.begin();
```

结果：

```
1  l=1
2  r=4
```

八、手写 lower_bound

1. 为什么要手写？

因为：

```
1  很多题不能直接套 STL。
```

而且：

```
1  手写后会真正理解边界。
```

2. 模板

```
1  int lower(vector<int> &a, int x)
2  {
3      int l=0, r=a.size();
4
5      while(l<r)
6      {
7          int mid=(l+r)/2;
8
9          if(a[mid]>=x)
10         {
11             r=mid;
12         }
13         else
14         {
15             l=mid+1;
16         }
17     }
18
19     return l;
20 }
```

3. 为什么 `r=mid` ?

因为：

```
1 mid 本身可能就是答案。
```

所以：

```
1 不能丢掉。
```

九、手写 `upper_bound`

```
1  int upper(vector<int> &a,int x)
2  {
3      int l=0,r=a.size();
4
5      while(l<r)
6      {
7          int mid=(l+r)/2;
8
9          if(a[mid]>x)
10         {
11             r=mid;
12         }
13         else
14         {
15             l=mid+1;
16         }
17     }
18
19     return l;
20 }
```

十、为什么边界二分常用 `while(l<r)` ?

因为：

```
1 边界二分是在“逼近答案位置”。
```

最后：

```
1 l==r
```

时：

```
1 答案唯一确定。
```

十一、答案二分

1. 什么是答案二分？

普通二分：

```
1 在数组中找数。
```

答案二分：

```
1 在“答案范围”中二分。
```

2. 例子

例如：

```
1 跳石头
```

目标：

```
1 最大化最小跳跃距离。
```

我们不知道答案。

但可以：

```
1 猜一个答案 mid。
```

然后：

- 1 检查能否做到。

十二、check 函数

1. check 是什么？

check(mid) :

- 1 判断答案 mid 是否可行。

2. 为什么能二分？

因为：

- 1 存在单调性。

例如：

- 1 距离越小越容易满足。

十三、答案二分模板

```
1  int l=0,r=1000000000;  
2  int ans=0;  
3  
4  while(l<=r)  
5  {  
6      int mid=(l+r)/2;  
7  
8      if(check(mid))  
9      {  
10         ans=mid;  
11         l=mid+1;  
12     }  
13     else  
14     {  
15         r=mid-1;
```

```
16     }  
17 }
```

十四、为什么这样移动？

1. 当前答案可行

如果：

```
1 mid 可以做到
```

说明：

```
1 也许还能更大。
```

所以：

```
1 l=mid+1;
```

2. 当前答案不可行

如果：

```
1 mid 做不到
```

说明：

```
1 答案必须更小。
```

所以：

```
1 r=mid-1;
```

十五、二分最核心的东西：单调性

1. 什么是单调性？

例如：

1 答案越小越容易成立。

或者：

1 答案越大越容易成立。

2. 为什么重要？

因为二分本质：

1 依赖“左边全对 / 右边全错”

或者：

1 左边全错 / 右边全对

十六、常见二分模型

模型	核心
普通二分	在有序数组中找值
边界二分	找第一个 / 最后一个满足条件的位置
lower_bound	第一个 $\geq x$
upper_bound	第一个 $> x$
答案二分	猜答案 + check
最大化最小值	跳石头类
最小化最大值	分段类

十七、Forbidden List2

1. 题目本质

给定：

1 禁止数字列表

询问：

1 第 y 个合法数字。

2. 核心思想

核心不是：

1 模拟找数。

而是：

1 统计某段区间有多少个禁止数字。

3. 为什么用二分？

因为：

1 禁止列表有序。

可以：

1 `upper_bound - lower_bound`

快速统计。

4. 本质模型

1 边界二分
2 +
3 有序数组统计

十八、常见错误

1. mid 写错

正确：

```
1  int mid=(l+r)/2;
```

2. 死循环

例如：

```
1  l=mid;
```

可能：

```
1  mid 永远不变。
```

通常：

```
1  l=mid+1;
```

或者：

```
1  r=mid-1;
```

3. 忘记排序

普通二分前：

```
1  sort(a,a+n);
```

4. lower_bound 返回的是迭代器

正确：

```
1
```

```
1 int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

5. 边界越界

例如：

```
1 pos==a.size()
```

时：

```
1 a[pos]
```

会越界。

6. check 没有单调性

如果：

```
1 答案不单调
```

就不能二分。

十九、本专题做过的题

题号	题目	类型	题面简述	对应知识点
P2249	查找	普通二分	在有序数组中找第一次出现位置	lower_bound
P1102	A-B 数对	排序 / 二分	统计满足条件的数对	lower_bound / upper_bound
P1873	砍树	答案二分	找锯子高度使木材满足要求	check
P1824	进击的奶牛	答案二分	最大化最小距离	最大化最小值
P2678	跳石头	答案二分	删除石头后最大化最小跳跃距离	check

题号	题目	类型	题面简述	对应知识点
P1182	数列分段 Section II	答案二分	最小化最大段和	最小化最大值
P3853	路标设置	答案二分	添加路标后最小化最大间距	check
P1577	切绳子	答案二分	求最大绳长	浮点二分
Forbidden List2	AtCoder	边界二分	禁止列表找合法数字	lower_bound

二十、本章最重要的结论

二分的核心不是模板。

真正重要的是：

- 1 每次排除一半答案。

普通二分：

- 1 依赖有序。

答案二分：

- 1 依赖单调性。

看到题目时：

先问自己：

- 1 能不能排除一半？
2 有没有单调性？

如果有：

- 1 就应该考虑二分。